DATA ENGINEERING PREPARATION

DAY 10 EXERCISES

 # LINUX PRACTICE

Linux Task 1 - Full pipeline in one session

Using the hospital_pipeline project folder.

Perform the following entirely using

terminal commands, no Python:

1. Display total line count of all three

raw CSV files in one command.

wc -l raw/*.csv

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ ls *.csv
appointments.csv  doctors.csv  employees.csv  patients.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ wc -l *.csv
  11 appointments.csv
   6 doctors.csv
   9 employees.csv
   9 patients.csv
  35 total
```

2. Extract unique specialties from doctors.csv.

cut + sort + uniq

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f3 doctors.csv | sort | uniq
Cardiology
Dermatology
General Medicine
Neurology
Orthopedics
specialty
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f5 appointments.csv | sort | uniq
Acne treatment
Back pain physiotherapy
Cardiac stress test
Epilepsy management
Fever and cold
Follow-up general checkup
Hypertension checkup
Knee pain evaluation
Migraine treatment
Routine vaccination
diagnosis
```

3. Find the highest fee in appointments.csv

without Python.

sort + head

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f6 appointments.csv | sort | head -1
150
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f6 appointments.csv | sort -nr | head -1
900
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$
```

4. Count how many appointments have

fee above 500.

Awk

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ awk -F',' 'NR>1 && $6 > 500 {count++} END {print count}' appointments.csv
4
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$
```

5. Extract appt_id, patient_id and fee

from appointments.csv,

sort by fee descending,

save to reports/high_fees.txt.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cut -d',' -f6,1,2 appointments.csv | sort -t',' -k3,3nr > high_fees.txt
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ ls -lrth
total 5.6M
-rwxrwxrwx 1 prashant prashant  62K Sep  8  2025  KODE-X_LINUX_DE.html
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cat high_fees.txt | column -t -s,
7         6           900
10        2           850
3         3           750
4         4           600
1         1           500
8         7           400
5         5           300
2         2           200
6         1           200
9         8           150
appt_id   patient_id  fee
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$
```

6. Search enriched_data.csv for all rows

where specialty is Cardiology.

Save to reports/cardiology_report.txt.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ awk -F',' 'NR==1 || $9=="Cardiology"' enriched_data.csv > reports/cardiology_repo
rt.txt
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ cd reports/
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/reports$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 141 Mar 28  2026 cardiology_report.txt
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/reports$ cat cardiology_report.txt | column -t -s,
  appt_id  appt_date  patient_id  patient_name  age  city  doctor_id  doctor_name  specialty  experience_years  diagnosis  fee  fee_c
ategory  day_of_week
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/reports$
```

Linux Task 2 - Archiving the pipeline

1. Confirm all output files are present

   in processed/ and reports/.

   ls -lh processed/ reports/

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/hospital_pipeline$ ls -lh processed/ reports/
processed/:
total 4.0K
-rwxrwxrwx 1 prashant prashant  657 Mar 27 21:40 appointments_clean.csv
-rwxrwxrwx 1 prashant prashant  269 Mar 27 08:30 doctors_clean.csv
-rwxrwxrwx 1 prashant prashant 1.3K Mar 28 10:48 enriched_data.csv
-rwxrwxrwx 1 prashant prashant  324 Mar 27 07:30 patients_clean.csv

reports/:
total 0
-rwxrwxrwx 1 prashant prashant 141 Mar 28  2026 cardiology_report.txt
-rwxrwxrwx 1 prashant prashant 104 Mar 28  2026 high_fees.txt
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/hospital_pipeline$
```

2. Create final archive:

   tar -czvf hospital_pipeline_final.tar.gz

   hospital_pipeline/

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ tar -cvzf hospital_pipeline_final.tar.gz hospital
_pipeline/
hospital_pipeline/
hospital_pipeline/archive/
hospital_pipeline/mini_projects.ipynb
hospital_pipeline/processed/
hospital_pipeline/processed/appointments_clean.csv
hospital_pipeline/processed/doctors_clean.csv
hospital_pipeline/processed/enriched_data.csv
hospital_pipeline/processed/patients_clean.csv
hospital_pipeline/raw/
hospital_pipeline/raw/appointments.csv
hospital_pipeline/raw/doctors.csv
hospital_pipeline/raw/patients.csv
hospital_pipeline/raw/profiler.ipynb
hospital_pipeline/reports/
hospital_pipeline/reports/cardiology_report.txt
hospital_pipeline/reports/high_fees.txt
```

3. Verify contents of the archive.

   tar -tvf hospital_pipeline_final.tar.gz

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ tar -tvf hospital_pipeline_final.tar.gz
drwxrwxrwx prashant/prashant     0 2026-03-29 09:07 hospital_pipeline/
drwxrwxrwx prashant/prashant     0 2026-03-23 22:12 hospital_pipeline/archive/
-rwxrwxrwx prashant/prashant 22023 2026-03-26 17:04 hospital_pipeline/mini_projects.ipynb
drwxrwxrwx prashant/prashant     0 2026-03-28 10:44 hospital_pipeline/processed/
-rwxrwxrwx prashant/prashant   657 2026-03-27 21:40 hospital_pipeline/processed/appointments_clean.csv
-rwxrwxrwx prashant/prashant   269 2026-03-27 08:30 hospital_pipeline/processed/doctors_clean.csv
-rwxrwxrwx prashant/prashant  1256 2026-03-28 10:48 hospital_pipeline/processed/enriched_data.csv
-rwxrwxrwx prashant/prashant   324 2026-03-27 07:30 hospital_pipeline/processed/patients_clean.csv
drwxrwxrwx prashant/prashant     0 2026-03-29 09:07 hospital_pipeline/raw/
-rwxrwxrwx prashant/prashant   465 2026-03-23 22:23 hospital_pipeline/raw/appointments.csv
-rwxrwxrwx prashant/prashant   206 2026-03-23 22:22 hospital_pipeline/raw/doctors.csv
-rwxrwxrwx prashant/prashant   256 2026-03-23 22:20 hospital_pipeline/raw/patients.csv
-rwxrwxrwx prashant/prashant 66667 2026-03-28 10:54 hospital_pipeline/raw/profiler.ipynb
drwxrwxrwx prashant/prashant     0 2026-03-29 09:06 hospital_pipeline/reports/
-rwxrwxrwx prashant/prashant   141 2026-03-28 22:10 hospital_pipeline/reports/cardiology_report.txt
-rwxrwxrwx prashant/prashant   104 2026-03-28 21:39 hospital_pipeline/reports/high_fees.txt
```

4. Check the archive file size.

   du -sh hospital_pipeline_final.tar.gz

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ du -sh hospital_pipeline_final.tar.gz
16K     hospital_pipeline_final.tar.gz
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$
```

5. Set permissions:

   chmod 644 on all .csv files

chmod 755 on all .py files

Verify with ls -l.

6. Write a one-line shell command that

counts how many .csv files exist

in the entire hospital_pipeline

directory tree.

find + wc

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/hospital_pipeline$ cd ..
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ find hospital_pipeline -type f -name "*.csv" |wc
-l
7
```

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ cd hospital_pipeline/
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/hospital_pipeline$ tree
.
├── archive
├── mini_projects.ipynb
├── processed
│   ├── appointments_clean.csv
│   ├── doctors_clean.csv
│   ├── enriched_data.csv
│   └── patients_clean.csv
├── raw
│   ├── appointments.csv
│   ├── doctors.csv
│   ├── patients.csv
│   └── profiler.ipynb
└── reports
    ├── cardiology_report.txt
    └── high_fees.txt

5 directories, 11 files
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/hospital_pipeline$
```

 # SQL PRACTICE

SQL Exercise 1 - Full hospital query set

Using patients, doctors, appointments tables.

1. Count total appointments per doctor.

   Show doctor_id and count.

   Order by count descending.

```
6
7   SELECT
8       DOCTOR_ID,
9       COUNT(*) AS TOTAL_APPT
10  FROM APPOINTMENTS
11  GROUP BY DOCTOR_ID
12  ORDER BY TOTAL_APPT DESC;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.005 seconds

	DOCTOR_ID	TOTAL_APPT
1	5	3
2	2	2
3	3	2
4	1	2

2. Find the doctor who collected the

   highest total fee.

   Show doctor_id and total_fee.

```
5   --    Show doctor_id and total_fee.
6   SELECT
7       DOCTOR_ID,
8       SUM(FEE) AS TOTAL_FEE
9   FROM APPOINTMENTS
10  GROUP BY DOCTOR_ID
11  ORDER BY TOTAL_FEE DESC
12  FETCH FIRST 1 ROW ONLY;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.001 seconds

	DOCTOR_ID	TOTAL_FEE
1	2	1450

3. Find patients who had more than

one appointment.

Show patient_id and appointment count.

```
4   --    Show patient_id and appointment count.
5   SELECT
6       PATIENT_ID,
7       COUNT(*) AS APPT_COUNT
8   FROM APPOINTMENTS
9   GROUP BY PATIENT_ID
10  HAVING COUNT(*) > 1;
11
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	PATIENT_ID	APPT_COUNT
1	1	2
2	2	2

4. Find the average fee per doctor.

Only show doctors with average fee

above 400.

```
4   --    above 400.
5   SELECT
6       DOCTOR_ID,
7       AVG(FEE) AS AVERAGE_FEE
8   FROM APPOINTMENTS
9       GROUP BY DOCTOR_ID
10      HAVING AVG(FEE) > 400;
11
```

Query result | Script output | DBMS output | Explain Plan | SQL history

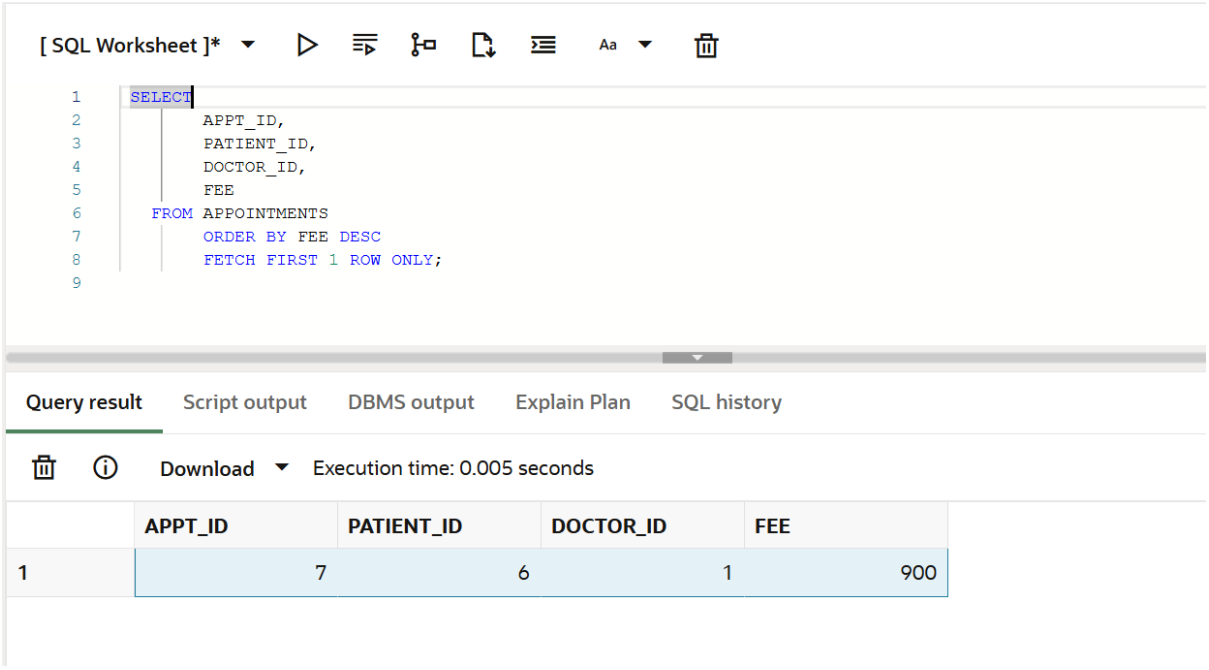
Download | Execution time: 0.001 seconds

	DOCTOR_ID	AVERAGE_FEE
1	1	700
2	2	725
3	3	575

5. Find the appointment with the highest fee.

Show appt_id, patient_id, doctor_id, fee.

Use FETCH FIRST 1 ROW ONLY.



SQL Exercise 2 - Business queries

from orders, products,customers tables

1. Find products that have never been ordered.

   Show product_id and product_name.

   Hint: Use NOT IN with a subquery

   on orders.product_id.

```
4   --    Hint: Use NOT IN with a subquery
5   --    on orders.product_id.
6   SELECT * FROM PRODUCTS;
7   SELECT
8       PRODUCT_ID,
9       PRODUCT_NAME
10  FROM PRODUCTS
11  WHERE PRODUCT_ID NOT IN (
12      SELECT PRODUCT_ID
13      FROM ORDERS
14  );
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.007 seconds

PRODUCT_ID	PRODUCT_NAME

No items to display.

2. Find customers who have placed

   more than 1 order.

   Show customer_id and order_count.

```
2   --    more than 1 order.
3   --    Show customer_id and order_count.
4   SELECT * FROM PRODUCTS;
5   SELECT * FROM ORDERS;
6   SELECT
7       CUSTOMER_ID,
8       COUNT(*) AS ORDER_COUNT
9   FROM ORDERS
10  GROUP BY CUSTOMER_ID
11  HAVING COUNT(*) > 1;
12
```

Query result | Script output | DBMS output | Explain Plan | SQL history

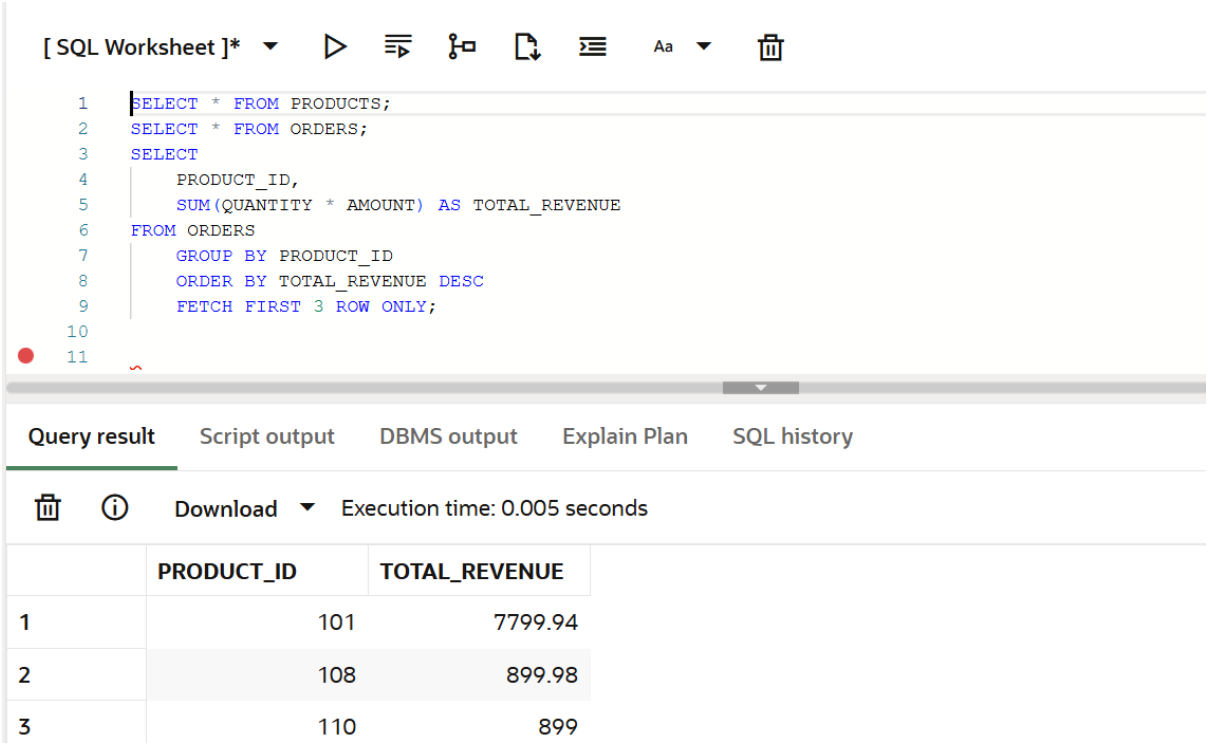
Download | Execution time: 0.004 seconds

	CUSTOMER_ID	ORDER_COUNT
1	206	2
2	203	2
3	204	2
4	202	3

3. Find the top 3 products by

total revenue (quantity x amount).

Show product_id and total_revenue.

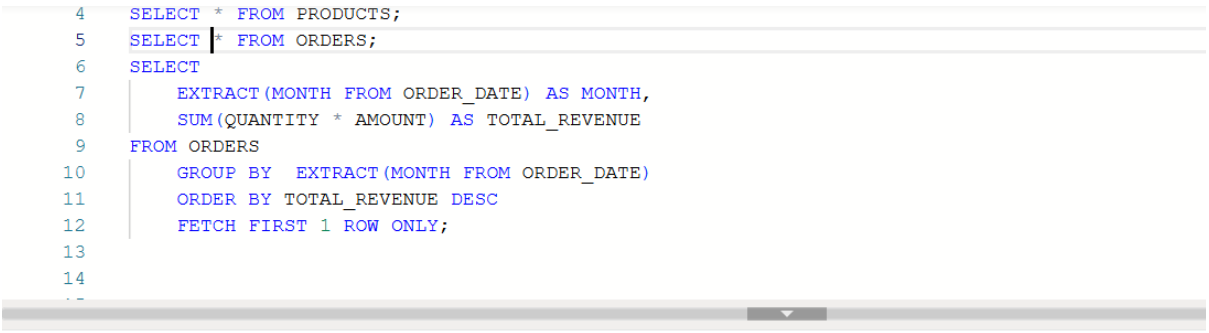


	PRODUCT_ID	TOTAL_REVENUE
1	101	7799.94
2	108	899.98
3	110	899

4. Find the month with the highest

total order revenue.

Hint: Use EXTRACT(MONTH FROM order_date).



	MONTH	TOTAL_REVENUE
1	1	12423.3

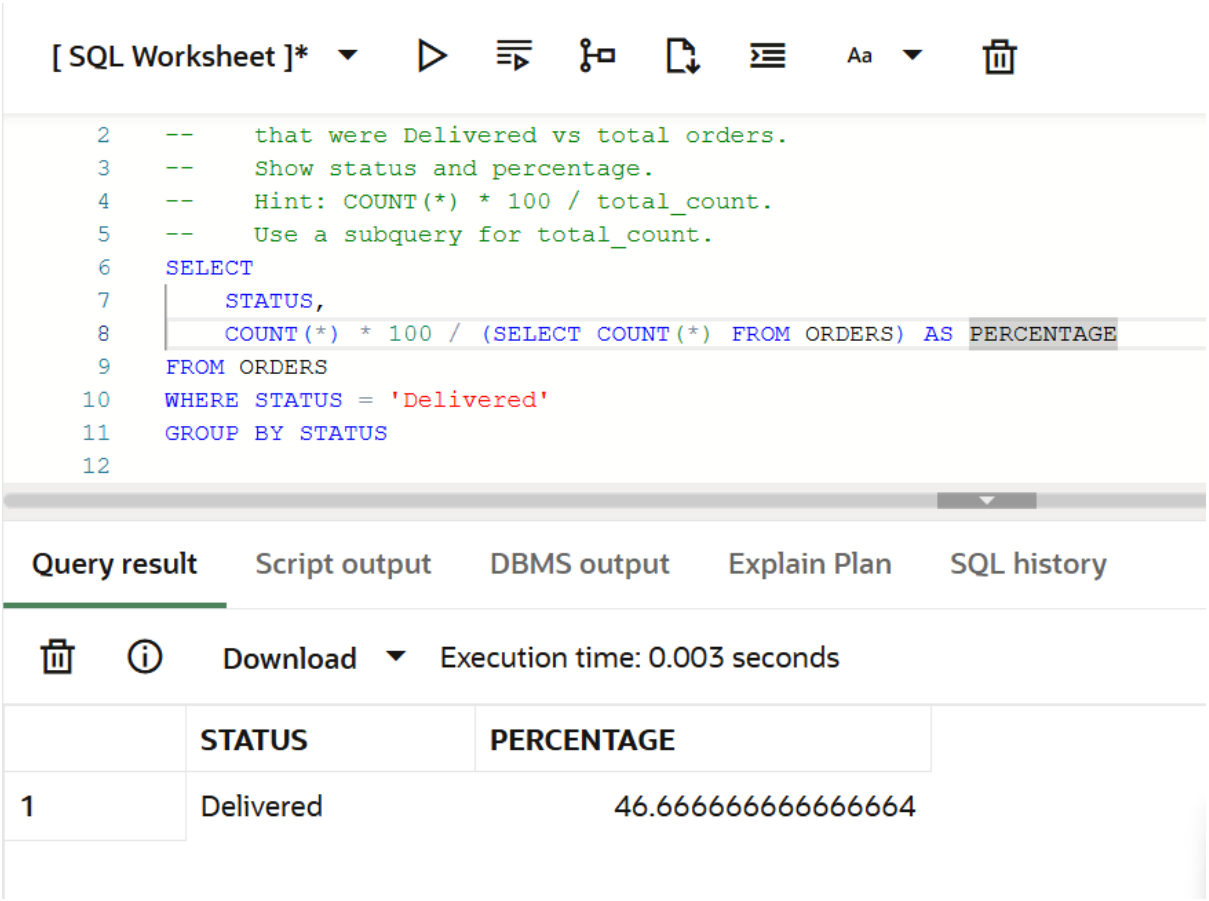
5. Find the percentage of orders

that were Delivered vs total orders.

Show status and percentage.

Hint: COUNT(*) * 100 / total_count.

Use a subquery for total_count.



SQL Exercise 3 - Advanced combined challenge

Write a single query for each:

1. From employees and department_info:

Show department name, employee count,

department budget, total salary,

and a column called budget_status:

total_salary > budget → "Over Budget"

total_salary <= budget → "Within Budget"

Use implicit join:

FROM employees e, department_info d

WHERE e.department = d.department

Group by department.

[SQL Worksheet]*

```
SELECT
    D.DEPARTMENT,
    COUNT(*) AS EMPLOYEE_COUNT,
    D.BUDGET,
    SUM(E.SALARY) AS TOTAL_SALARY,
    CASE
        WHEN SUM(E.SALARY) > D.BUDGET THEN 'Over Budget'
        ELSE 'Within Budget'
    END AS BUDGET_STATUS
FROM EMPLOYEES E, DEPARTMENT_INFO D
    WHERE E.DEPARTMENT = D.DEPARTMENT
    GROUP BY D.DEPARTMENT, D.BUDGET;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.011 seconds

	DEPARTMENT	EMPLOYEE_COUNT	BUDGET	TOTAL_SALARY	BUDGET_STATUS
2	Finance	3	600000	219000	Within Budget
3	Engineering	4	1500000	360000	Within Budget
4	HR	2	400000	114000	Within Budget

3. From appointments:

For each patient_id show:

- total appointments

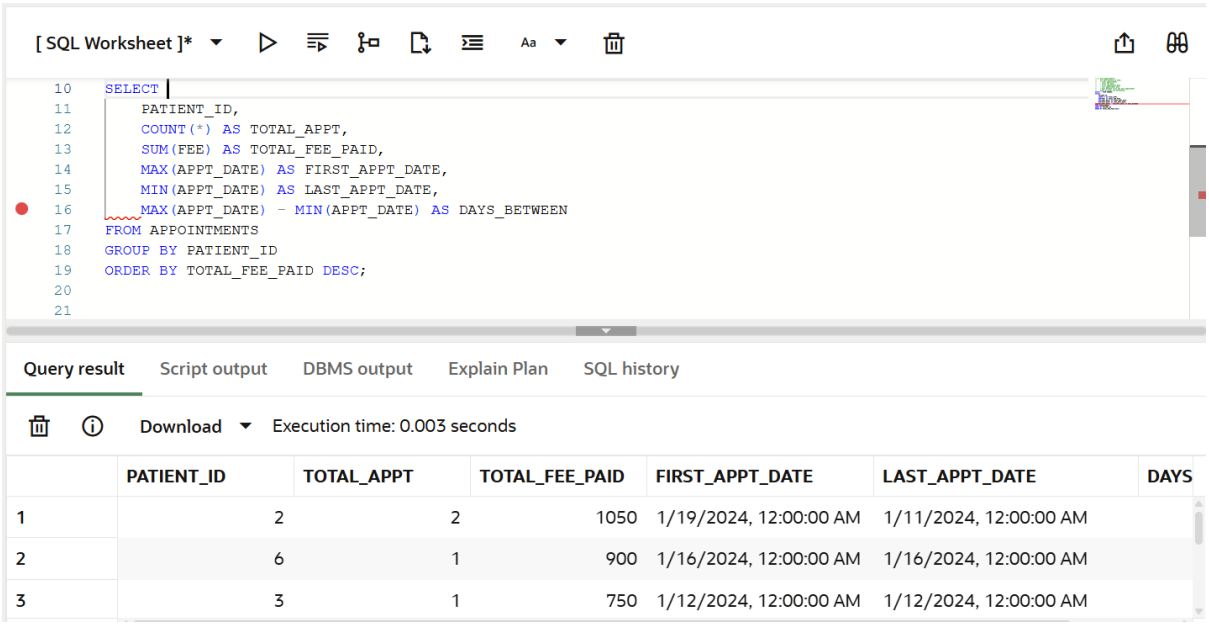
- total fee paid

- first appointment date

- last appointment date

- days between first and last appointment

Order by total fee descending.



	PATIENT_ID	TOTAL_APPT	TOTAL_FEE_PAID	FIRST_APPT_DATE	LAST_APPT_DATE	DAYS
1	2	2	1050	1/19/2024, 12:00:00 AM	1/11/2024, 12:00:00 AM	
2	6	1	900	1/16/2024, 12:00:00 AM	1/16/2024, 12:00:00 AM	
3	3	1	750	1/12/2024, 12:00:00 AM	1/12/2024, 12:00:00 AM	

 # Exercise 1 - Pandas: Advanced groupby and agg

Load enriched_data.csv from the Day 9

hospital project (or recreate it).

1. Using groupby on specialty:

   Show count, total fee, avg fee, max fee.

   Use .agg({'fee': ['count','sum','mean','max']})

   Rename columns cleanly after aggregating.

```
#    Use .agg({'fee': ['count','sum','mean','max']})
#    Rename columns cleanly after aggregating.
import pandas as pd
df = pd.read_csv("enriched_data.csv")

# total_fee = df.groupby("specialty")["fee"].sum()
# print(total_fee)

# average_fee = df.groupby("specialty")["fee"].mean()
# print(average_fee)

# max_fee = df.groupby("specialty")["fee"].max()
# print(max_fee)

result = (
    df.groupby("specialty").agg({'fee': ['count', 'sum', 'mean', 'max']})
)
result.columns = ['count', 'total_fee', 'average_fee', 'max_fee']
result = result.reset_index()
print(result)
```

[8] ✓ 0.0s

```
           specialty  count  total_fee  average_fee  max_fee
0        Cardiology      2       1400   700.000000      900
1       Dermatology      1        300   300.000000      300
2  General Medicine      3        550   183.333333      200
3         Neurology      2       1450   725.000000      850
4       Orthopedics      2       1150   575.000000      750
```

2. Using groupby on age_group:

   Show count of patients and avg fee.

   Sort by avg fee descending.

```python
# 2. Using groupby on age_group:
#    Show count of patients and avg fee.
#    Sort by avg fee descending.
result1 = (
    df.groupby("age").agg({'fee': ['count', 'mean']})
)
result1.columns = ['count', 'average_fee']
result1 = result1.sort_values(by='average_fee', ascending=False)
result1 = result1.reset_index()
print(result1)
```

✓ 0.0s

```
   age  count  average_fee
0   56      1        900.0
1   46      1        750.0
2   29      1        600.0
3   39      2        525.0
4   32      1        400.0
5   34      2        350.0
6   24      1        300.0
7   19      1        150.0
```

3. Using groupby on day_of_week:

Show total fee per day.

Which day generated the most revenue?

```python
# 3. Using groupby on day_of_week:
#    Show total fee per day.
#    Which day generated the most revenue?
total_fee = (
    df.groupby("day_of_week")
      .agg(total_fee = ('fee', 'sum'))
      .sort_values(by='total_fee', ascending=False)
      .reset_index()
)
print(total_fee)

top = total_fee.iloc[0]
print("Highest revenue day:")
print(top)
```

✓ 0.0s

```
  day_of_week  total_fee
0      Friday       1600
1     Tuesday        900
2   Wednesday        900
3    Saturday        600
4    Thursday        350
5      Sunday        300
6      Monday        200
Highest revenue day:
day_of_week    Friday
total_fee        1600
Name: 0, dtype: object
```

4. Using groupby on fee_category:

Show count and avg patient age.

Is there a pattern?

```
# 4. Using groupby on fee_category:
#    Show count and avg patient age.
#    Is there a pattern?
result = (
    df.groupby("fee_category").agg({'age':['count', 'mean']})
)
result.columns = ['Count', 'Average_patient_age']
result = result.reset_index()
print(result)
```

✓ 0.0s

```
  fee_category  Count  Average_patient_age
0         High      3            47.000000
1          Low      4            29.000000
2       Medium      3            31.666667
```

5. Cross-tabulation:

Count how many appointments each

specialty handled per fee_category.

Use pd.crosstab() or groupby on two columns.

```
# 5. Cross-tabulation:
#    Count how many appointments each
#    specialty handled per fee_category.
#    Use pd.crosstab() or groupby on two columns.

# flow :- Input columns → find unique categories → count combinations → create matrix

cross_tabulation = pd.crosstab(
    df["specialty"],
    df["fee_category"]
)
print(cross_tabulation)

cross_tabulation1 = pd.crosstab(
    df["patient_name"],
    df["day_of_week"]
)
print(cross_tabulation1)
```

✓ 0.0s

```
fee_category      High  Low  Medium
specialty
Cardiology           1    0       1
Dermatology          0    1       0
General Medicine     0    3       0
Neurology            1    0       1
Orthopedics          1    0       1
```

Exercise 2 - Nested JSON handling

Write code to:

1. Load the JSON file.

```
# Write code to:
# 1. Load the JSON file.
import pandas as pd
df = pd.read_json("hospital_report.json")
print(df)
```

✓ 0.0s

```
  report_date      hospital                                        departments
0  2024-01-20  City General  {'name': 'Cardiology', 'doctor': 'Dr. Anil Meh...
1  2024-01-20  City General  {'name': 'General Medicine', 'doctor': 'Dr. Ca...
```

2. Print report_date and hospital name.

```
print("=========================================")
# 2. Print report_date and hospital name.
print(f"{df["report_date"]} \n{df["hospital"]}")
```

✓ 0.0s

```
=========================================
0    2024-01-20
1    2024-01-20
Name: report_date, dtype: object
0    City General
1    City General
Name: hospital, dtype: object
```

3. Loop through each department and print:

- Department name
- Doctor name
- Number of appointments
- Total fee collected

```python
# 3. Loop through each department and print:
#    - Department name
#    - Doctor name
#    - Number of appointments
#    - Total fee collected
for i in raw_data["departments"]:
    dept_name = i["name"]
    doctor_name = i["doctor"]

    appointment = i["appointments"]
    num_appointment = len("appointment")
    total_fee = sum([appt["fee"] for appt in appointment])

    print(f"Department Name: {dept_name}")
    print(f"Doctor Name: {doctor_name}")
    print(f"Number Of Appointment: {num_appointment}")
    print(f"Total Fee Collected: {total_fee}")
    print("="*50)
```

✓ 0.0s

```
Department Name: Cardiology
Doctor Name: Dr. Anil Mehta
Number Of Appointment: 11
Total Fee Collected: 1400
==================================================
Department Name: General Medicine
Doctor Name: Dr. Carlos Torres
Number Of Appointment: 11
```

4. Flatten the nested structure into a

pandas DataFrame with columns:

department, doctor, appt_id,

patient, fee, date.

Do this without using json_normalize

— use a loop and list of dicts.

```python
# 4. Flatten the nested structure into a
rows = []
for dept in raw_data["departments"]:
    dept_name = dept["name"]
    doctor_name = dept["doctor"]

    for appt in dept["appointments"]:
        row ={
            "department_name": dept_name,
            "doctor_name": doctor_name,
            "appt_id": appt["appt_id"],
            "patient_name": appt["patient"],
            "fee": appt["fee"],
            "date": appt["fee"]
        }
        rows.append(row)

df = pd.DataFrame(rows)
print(df.head())
print(df.shape)
```

✓ 0.0s

```
    department_name        doctor_name  appt_id  patient_name  fee  date
0        Cardiology     Dr. Anil Mehta        1     Raj Kumar  500   500
1        Cardiology     Dr. Anil Mehta        7  Ana Gonzalez  900   900
2  General Medicine  Dr. Carlos Torres        2   Sneha Patel  200   200
3  General Medicine  Dr. Carlos Torres        6     Raj Kumar  200   200
4  General Medicine  Dr. Carlos Torres        9  Emily Carter  150   150
(5, 6)
```

5. Find which department earned more.

```
# 5. Find which department earned more.
'''
# Difference Between max() and idxmax()
| Function   | Returns                              |
| ---------- | ------------------------------------ |
| max()      | Maximum value → 1400                 |
| idxmax()   | Label of max value → 'Cardiology'    |
'''
enrned_more = df.groupby("department_name")["fee"].sum().idxmax()
print(f"earned more department name is: {enrned_more}")
```

✓ 0.0s

```
earned more department name is: Cardiology
```

6. Export flattened DataFrame to

department_report.csv.

```
# 6. Export flattened DataFrame to
#    department_report.csv.
df.to_csv("department_report.csv")
df.head()
```

✓ 0.0s

	department_name	doctor_name	appt_id	patient_name	fee	date
0	Cardiology	Dr. Anil Mehta	1	Raj Kumar	500	500
1	Cardiology	Dr. Anil Mehta	7	Ana Gonzalez	900	900
2	General Medicine	Dr. Carlos Torres	2	Sneha Patel	200	200
3	General Medicine	Dr. Carlos Torres	6	Raj Kumar	200	200
4	General Medicine	Dr. Carlos Torres	9	Emily Carter	150	150

 # Exercise 3 - Exception handling with file and data operations

Write a function called safe_load_csv(filepath)

that:

- Tries to load the file using pandas

- Handles FileNotFoundError with a clear message

- Handles pd.errors.EmptyDataError

- Handles any other exception with a generic

error message

- Returns the DataFrame on success,

None on failure

```python
# Write a function called safe_load_csv(filepath)
# that:
# - Tries to load the file using pandas
# - Handles FileNotFoundError with a clear message
# - Handles pd.errors.EmptyDataError
# - Handles any other exception with a generic
#   error message
# - Returns the DataFrame on success,
#   None on failure
def safe_load_csv(filepath):
    try:
        df = pd.read_csv(filepath)
        return df
    except FileNotFoundError:
        print(f"Error: File Not Found {filepath}")
    except pd.errors.EmptyDataError:
        print(f"Error: File is Empty {filepath}")
    except Exception as e:
        print(f"Error: Unable to load a file {filepath}")
        print(f"Details: {e}")
    return None

df = safe_load_csv("departments_report.csv")
```

✓ 0.0s

```
Error: File Not Found departments_report.csv
Loading failed.
```

```
   Unnamed: 0   department_name        doctor_name  appt_id  patient_name  \
0           0        Cardiology     Dr. Anil Mehta        1     Raj Kumar
1           1        Cardiology     Dr. Anil Mehta        7  Ana Gonzalez
2           2  General Medicine  Dr. Carlos Torres        2   Sneha Patel
3           3  General Medicine  Dr. Carlos Torres        6     Raj Kumar
4           4  General Medicine  Dr. Carlos Torres        9  Emily Carter

   fee  date
0  500   500
1  900   900
2  200   200
3  200   200
4  150   150
========================================
Error: File Not Found 'data.csv'
None
========================================
Error: Given File is Empty 'empty.csv'
========================================
Error: Unable to load a file 'None'
Details: Invalid file path or buffer object type: <class 'NoneType'>
```

Write a function called safe_process(df, column)

that:

- Tries to compute mean of the given column

- Handles KeyError if column does not exist

- Handles TypeError if column is not numeric

- Returns the mean or None

```
# Write a function called safe_process(df, column)
# that:
# - Tries to compute mean of the given column
# - Handles KeyError if column does not exist
# - Handles TypeError if column is not numeric
# - Returns the mean or None
# KeyError in Python
    # KeyError is raised when you try to access a key that does not exist in a dictionary
    # or similar data structure (like a pandas DataFrame column).

def safe_process(df, column):
    try:
        data = df[column]

        mean_value = data.mean()

        return mean_value

    except KeyError:
        print(f"Error: \'{column}\' Column does not exist.")
    except TypeError:
        print(f"Error: \'{column}\' Column is not numeric.")
    return None
safe_process(df, "salary")
```

 # Exercise 4 - Modular code structure

Refactor the following into a clean module.

Create a file called data_utils.py with

these functions:

1. load_csv(filepath) — loads and returns df

2. clean_strings(df) — strips all string cols

3. add_fee_category(df, col='fee') — adds

fee_category column using cut or apply

4. summarise(df, group_col, agg_col) — groups

by group_col and returns sum and mean

of agg_col

5. export_csv(df, filepath) — saves without index

Then create main.py that:

- Imports from data_utils

- Calls each function in sequence on

appointments.csv

- Prints summary and exports result

This simulates how real Python pipeline

code is organized in production.

list > data_utils.py > ...

```python
import pandas as pd

# 1. load_csv(filepath) — loads and returns df
def load_csv(filepath):
    return pd.read_csv(filepath)

# 2. clean_strings(df) — strips all string cols
def clean_strings(df):
    for column in df.select_dtypes(include="object").column:
        df[column] = df[column].str.strip()
    return df

# 3. add_fee_category(df, col='fee') — adds
def add_fee_category(df, column="fee"):
    bins = [0, 400, 750, float('inf')]
    labels = ["Low", "Medium", "High"]

    df["fee_category"] = pd.cut(df[column], bins=bins, labels=labels)
    return df

# 4. summarise(df, group_col, agg_col) — groups
#    by group_col and returns sum and mean
#    of agg_col
def summarise(df, group_col, agg_col):
    summary = df.groupby(group_col)[agg_col].agg(['sum', 'mean'])
    return summary

# 5. export_csv(df, filepath) — saves without index
def export_csv(df, filepath):
    df.to_csv(filepath, index=False)
```

list > main.py > main

```python
from data_utils import (
    load_csv,
    clean_strings,
    add_fee_category,
    summarise,
    export_csv
)
def main():
    df = load_csv("appointments.csv")

    df = clean_strings(df)

    df = add_fee_category(df, column='fee')

    summary = summarise(df, group_col="department", agg_col="fee")

    print("Summary")
    print(summary)

    export_csv(df, "processed_appointments.csv")

if __name__ == "__main__":
    main()
```

 # Exercise 5 - Pandas: Date operations

Load appointments_clean.csv from Day 9.

The appt_date column should already be datetime.

If not, convert it using pd.to_datetime().

1. Extract year, month, day into

separate columns.

```
# 1. Extract year, month, day into
#    separate columns.

import pandas as pd

df = pd.read_csv("appointments_clean.csv")
# print(df)
df["appt_date"] = pd.to_datetime(df["appt_date"], errors="coerce")

df["year"] = df["appt_date"].dt.year
df["month"] = df["appt_date"].dt.month
df["day"] = df["appt_date"].dt.day
print(df[['appt_date', 'year', 'month', 'day']])
```

```
   appt_date  year  month  day
0 2024-01-10  2024      1   10
1 2024-01-11  2024      1   11
2 2024-01-12  2024      1   12
3 2024-01-13  2024      1   13
4 2024-01-14  2024      1   14
5 2024-01-15  2024      1   15
6 2024-01-16  2024      1   16
7 2024-01-17  2024      1   17
8 2024-01-18  2024      1   18
9 2024-01-19  2024      1   19
```

2. Extract day of week name.

```
# 2. Extract day of week name.
df["day_name"] = df["appt_date"].dt.day_name()
print(df[['day_name']])
```

```
    day_name
0  Wednesday
1   Thursday
2     Friday
3   Saturday
4     Sunday
5     Monday
6    Tuesday
7  Wednesday
8   Thursday
9     Friday
```

3. Find which month had the most appointments.

```
# 3. Find which month had the most appointments.

count_appointment = df["month"].value_counts()
# count_appointment = df["month"].value_counts()
most_appointment = count_appointment.idxmax()
print(f"Month had the most appointments is : {most_appointment}")
print("Count of appointment:")
print(count_appointment)
```

✓ 0.0s

```
Month had the most appointments is : 1
Count of appointment:
month
1    10
Name: count, dtype: int64
```

4. Calculate days elapsed since each appointment

to today using pd.Timestamp.today().

```
# 4. Calculate days elapsed since each appointment
#    to today using pd.Timestamp.today().

today = pd.Timestamp.today()
df["days_elapsed"] = (today - df["appt_date"]).dt.days
print(df[['appt_date', 'days_elapsed']])
```

✓ 0.0s

```
app_date day_elapseddays
   appt_date  days_elapsed
0 2024-01-10           813
1 2024-01-11           812
2 2024-01-12           811
3 2024-01-13           810
4 2024-01-14           809
5 2024-01-15           808
6 2024-01-16           807
7 2024-01-17           806
8 2024-01-18           805
9 2024-01-19           804
```

5. Filter appointments that happened

in the first 10 days of January 2024.

```
# 5. Filter appointments that happened
#    in the first 10 days of January 2024.

filtered_df = df[
    (df["appt_date"] >= "2024-01-01") &
    (df["appt_date"] <= "2024-01-10")
]
print(filtered_df.head())
```

```
   Unnamed: 0  appt_id  patient_id  doctor_id  appt_date  \
0           0        1           1          1 2024-01-10

             diagnosis  fee fee_category day_of_week  year  month  day  \
0  Hypertension checkup  500       Medium   Wednesday  2024      1   10

    day_name  days_elapsed
0  Wednesday           813
```

6. Sort appointments by date ascending

and reset the index.

```
# 6. Sort appointments by date ascending
#    and reset the index.
sorted = df.sort_values(by="appt_date").reset_index(drop=True)
sorted.head(10)
```

Python

	Unnamed: 0	appt_id	patient_id	doctor_id	appt_date	diagnosis	fee	fee_category	day_of_week	year	month	day	day_name	days_elapsed
0	0	1	1	1	2024-01-10	Hypertension checkup	500	Medium	Wednesday	2024	1	10	Wednesday	813
1	1	2	2	5	2024-01-11	Fever and cold	200	Low	Thursday	2024	1	11	Thursday	812
2	2	3	3	3	2024-01-12	Knee pain evaluation	750	High	Friday	2024	1	12	Friday	811
3	3	4	4	2	2024-01-13	Migraine treatment	600	Medium	Saturday	2024	1	13	Saturday	810
4	4	5	5	4	2024-01-14	Acne treatment	300	Low	Sunday	2024	1	14	Sunday	809
5	5	6	1	5	2024-01-15	Follow-up general checkup	200	Low	Monday	2024	1	15	Monday	808
6	6	7	6	1	2024-01-16	Cardiac stress test	900	High	Tuesday	2024	1	16	Tuesday	807
7	7	8	7	3	2024-01-17	Back pain physiotherapy	400	Medium	Wednesday	2024	1	17	Wednesday	806
8	8	9	8	5	2024-01-18	Routine vaccination	150	Low	Thursday	2024	1	18	Thursday	805

Concepts: dt.year, dt.month, dt.day,

dt.day_name(), pd.Timestamp.today(),

timedelta arithmetic, sort_values()j111`` qq